ARQUITETURA DE

SISTEMAS

E-COMMERCE · PARTE 2: API GATEWAY E PEDIDOS

Prof. Augusto Gehrke

CONTEÚDO

1. O problema: o cliente falando com cada serviço
2. O que um API Gateway faz
3. Um gateway escrito do zero
4. Kong: por dentro, plugins e demonstração ao vivo
5. Borda × rede interna; o kong.yml do e-commerce
6. O serviço de pedidos e a chamada síncrona
7. O que entregar

HOJE: UMA PORTA SÓ

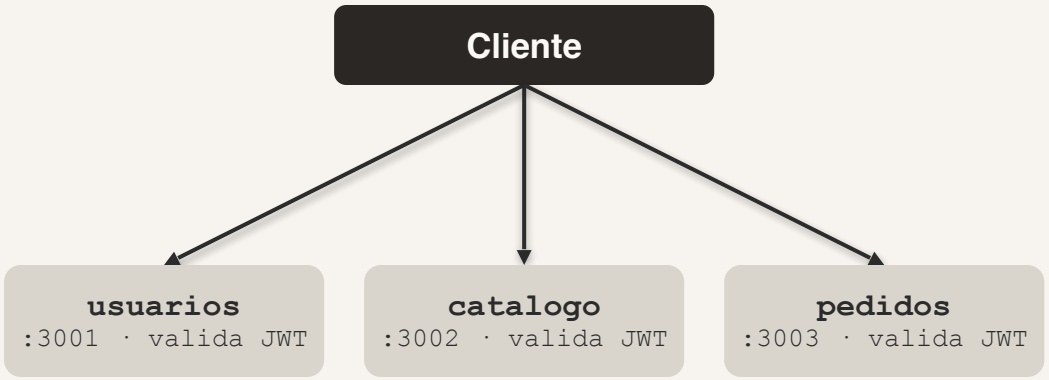
Na aula passada, cada serviço tinha a sua porta e o catálogo deixava qualquer um criar produto. Hoje isso acaba.

1. **Entender o gateway por dentro**, lendo um feito do zero (umas 100 linhas)
2. **Colocar o Kong** na frente de tudo: rotas, JWT e perfis num arquivo só
3. **Criar o serviço** `pedidos`: carrinho e checkout
4. **Fechar as portas** 3001 e 3002: só o gateway fica exposto

CONTINUA EM DUPLA, no mesmo repositório da aula passada.

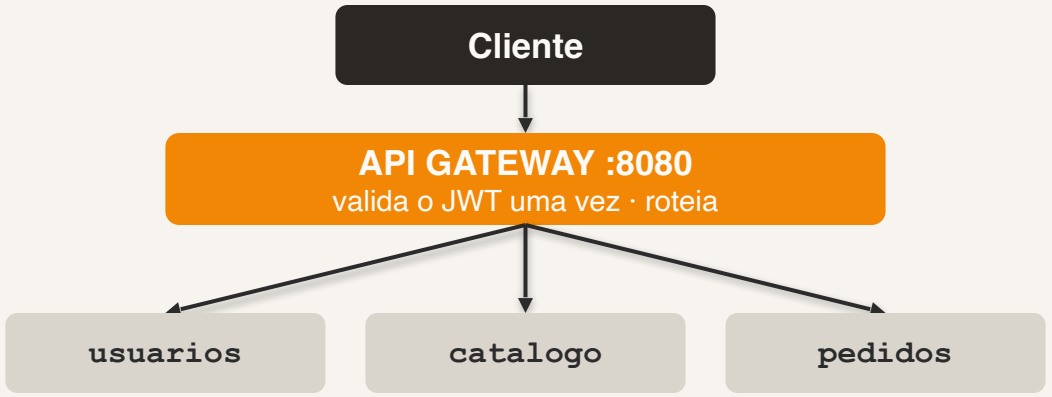
SEM GATEWAY × COM GATEWAY

SEM GATEWAY



- O cliente precisa conhecer o endereço de cada serviço.
- Cada serviço valida o token e guarda o segredo.
- Toda porta publicada é uma porta para atacar.
- Trocar um serviço de lugar quebra o cliente.

COM GATEWAY



- Um endereço só. O resto fica na rede interna.
- O token é validado num lugar só.
- Os serviços recebem a identidade pronta.
- Dá para mover, escalar e trocar serviços sem o cliente perceber.

O QUE UM API GATEWAY FAZ

É o porteiro do sistema: toda requisição de fora passa por ele antes de chegar a um serviço.

ROTEAR

Olha o método e o caminho e decide para qual serviço vai: /produtos → catalogo.

AUTENTICAR

Valida o JWT. Sem token ou com token falso: 401, e o serviço nem fica sabendo.

AUTORIZAR A ROTA

Rota de admin e o perfil é cliente: 403.

IDENTIDADE

Injeta X-User-Id, X-User-Email e X-User-Role para o serviço.

RASTREAR

Dá um X-Request-Id a cada requisição, que segue por todos os serviços.

E mais, nos gateways prontos: limite de requisições, cache, balanceamento, métricas, HTTPS, CORS.

QUEM DECIDE O QUÊ

GATEWAY: A PORTA

Quem é você? O token é válido? (401)

Esta rota é só de admin, e você é admin? (403)

Não conhece as regras de negócio. Não sabe o que é um pedido.

SERVIÇO: O RECURSO

Este pedido é seu? Este endereço é seu? (404)

Este pedido ainda pode ser cancelado? (409)

Confia nos cabeçalhos X-User-* porque só o gateway alcança o serviço.

Por que 404 e não 403 no pedido de outra pessoa? Um 403 confirma que o pedido existe. O 404 não entrega nem essa informação.

REVISÃO: O JWT DA LOJA

Três partes separadas por ponto: **cabeçalho** . **dados (claims)** . **assinatura**. Os dados são só Base64: qualquer um lê, ninguém altera.

```
// payload do token emitido por usuarios
{
  "sub":    "05be7f0d-cb73-...",  // id
  "email":  "cliente@loja.com",
  "perfil": "cliente",  // veio do banco
  "iat":    1791600000,
  "exp":    1791607200     // vence em 2 h
}
```

A assinatura é feita com o JWT_SECRET (HS256). Mudou uma letra do payload, a assinatura não bate.

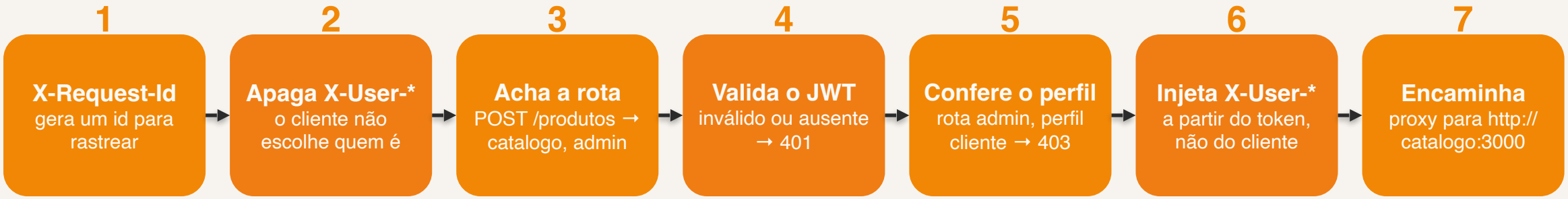
Quem tem o segredo consegue validar e também emitir tokens. Hoje: usuarios e o gateway.

Novo hoje: o Kong usa o campo `perfil` para decidir quem pode entrar nas rotas de admin.

Nunca coloque no token senha, CPF ou cartão: o payload não é criptografado, só assinado.

O CAMINHO DE UMA REQUISIÇÃO

```
POST /produtos   Authorization: Bearer eyJhbG...   X-User-Role: admin   // tentativa
```



Passos 1 a 6: o gateway responde sozinho se algo der errado. O serviço não recebe nada.

Passo 7: o catalogo recebe a requisição já com `X-User-Id`, `X-User-Role` e `X-Request-Id`. Ele não precisa saber o que é JWT.

Se o catalogo estiver fora do ar: o gateway responde 502 ou 503 em vez de deixar o cliente esperando.

PARTE 1: UM GATEWAY DO ZERO

Antes de usar um gateway pronto, vamos ler um escrito à mão, em Node e Express.

São umas 100 linhas, com os 7 passos do slide anterior. Depois de entender cada um, fica fácil reconhecer o mesmo trabalho no Kong.

NO REPOSITÓRIO DE REFERÊNCIA: `gateway-do-zero/src/index.js`

DO ZERO: A TABELA DE ROTAS

```
// A primeira regra que casar vale. metodo ausente = qualquer método.
const REGRAS = [
  { metodo: 'POST', caminho: /^\/auth\/login$/,          servico: 'usuarios', acesso: 'publico' },
  { metodo: 'POST', caminho: /^\/usuarios$/,            servico: 'usuarios', acesso: 'publico' },
  { metodo: 'GET',  caminho: /^\/usuarios$/,            servico: 'usuarios', acesso: 'admin' },
  {                caminho: /^\/usuarios\/me(\/.*)?$/, servico: 'usuarios', acesso: 'logado' },
  { metodo: 'GET',  caminho: /^\/(produtos|categorias)(\/.*)?$/, servico: 'catalogo', acesso: 'publico' },
  {                caminho: /^\/(produtos|categorias)(\/.*)?$/, servico: 'catalogo', acesso: 'admin' },
  {                caminho: /^\/(carrinho|pedidos)(\/.*)?$/, servico: 'pedidos', acesso: 'logado' },
];
```

- ✓ **A ordem importa:** GET /produtos casa com a regra pública antes de cair na regra de admin.
- ✓ **Três níveis de acesso:** `publico` (sem token), `logado` (qualquer token válido), `admin` (token com perfil admin).
- ✓ **Rota que não está na tabela:** 404 no próprio gateway.

DO ZERO: O PORTEIRO

```
app.use((req, res, next) => {
  // 1. id para rastrear
  req.headers['x-request-id'] ||= crypto.randomUUID();
  // 2. ninguém de fora escolhe quem é
  delete req.headers['x-user-role']; // e id, email
  // 3. qual regra?
  const regra = REGRAS.find(r => casa(r, req));
  if (!regra) return res.status(404)...;
  // 4. valida o token
  let usuario = null;
  try { usuario = jwt.verify(token, JWT_SECRET); }
  catch {}
  if (regra.acesso !== 'publico' && !usuario)
    return res.status(401).json({ erro: 'Faça login...' });
  // 5. perfil
  if (regra.acesso === 'admin' && usuario.perfil !== 'admin')
    return res.status(403).json(...);
  // 6. identidade vinda do token
  req.headers['x-user-id'] = usuario.sub; // email, role
  req.servico = regra.servico;
  next();
});
```

Simplificado para caber no slide. O arquivo completo está no repositório de referência.

Passo 2 antes do 6: primeiro apaga o que veio de fora, depois escreve o que veio do token.

Teste: mande `X-User-Role: admin` sem token para GET /usuarios. Sem o passo 2, o serviço acreditaria.

Repare: o gateway não abre o banco de ninguém. Ele só lê o token e a tabela de regras.

DO ZERO: ENCAMINHAR (PROXY)

```
const SERVICOS = {
  usuarios: 'http://usuarios:3000', catalogo: 'http://catalogo:3000',
  pedidos: 'http://pedidos:3000',
};

app.use(createProxyMiddleware({
  router: req => SERVICOS[req.servico],  // o serviço que a regra escolheu
  on: { error: (err, req, res) => res.status(502).json({ erro: 'Serviço indisponível.' }) },
}));
```

- ✓ **Os nomes** `usuarios` e `catalogo` são o DNS da rede do compose. Ninguém de fora resolve esses nomes.
- ✓ **O proxy repassa** método, caminho, query, corpo e cabeçalhos. A resposta do serviço volta igual ao cliente.
- ✓ **Biblioteca:** http-proxy-middleware. O gateway todo usa express, jsonwebtoken e ela.

POR QUE NÃO USAR O NOSSO?

Escrever do zero ensina como funciona. Em produção, quase ninguém mantém o próprio gateway. Faltaria:

- Limite de requisições por cliente (rate limiting)
- Balanceamento com verificação de saúde dos serviços
- Cache de respostas, CORS, HTTPS, mTLS
- Métricas, logs para vários destinos, painel de gestão
- Anos de correções de segurança e desempenho

PRONTOS: Kong, Traefik, NGINX, Envoy, AWS API Gateway, Azure API Management.

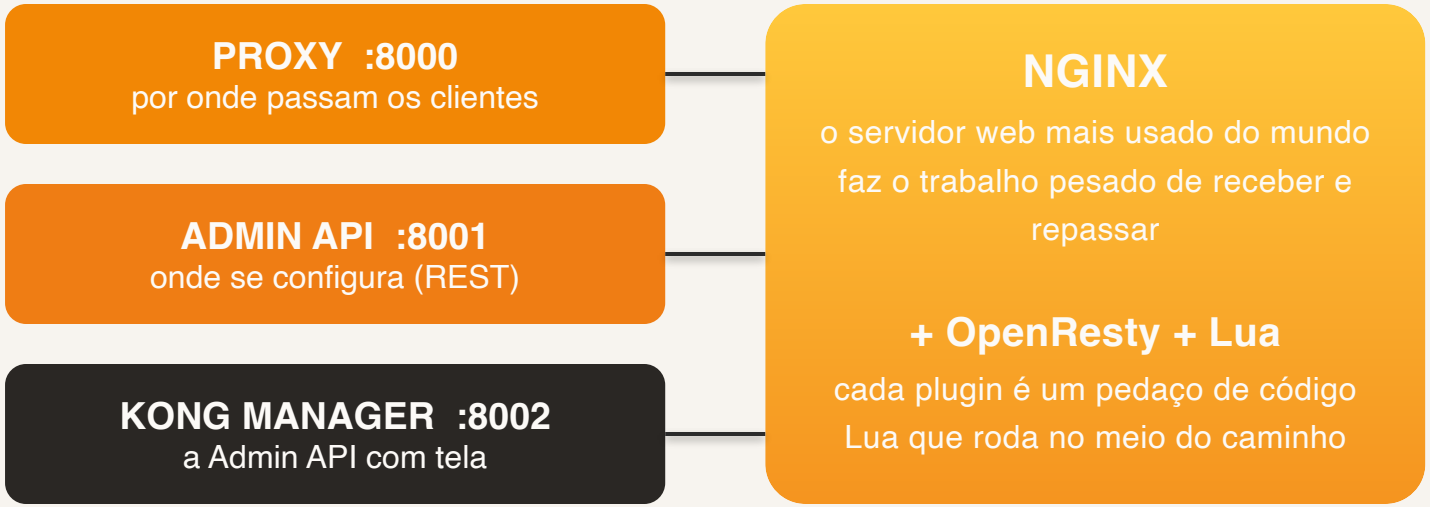
PARTE 2: KONG

Gateway de código aberto, construído sobre o NGINX. Usaremos o modo sem banco: toda a configuração num arquivo, kong.yml, versionado no repositório.

Peça	O que é	No nosso gateway do zero
service	Para onde vai: um microserviço e a sua URL	o objeto SERVICOS
route	Quais métodos e caminhos levam a um service	cada linha de REGRAS
plugin	Algo que acontece no caminho: JWT, rate limit, logs	os passos 1, 2, 4, 5 e 6 do middleware
consumer	Quem chama a API, com as suas credenciais	não existia: aqui, um por perfil
tag	Uma etiqueta livre em qualquer peça	o campo acesso: publico, logado, admin

Mesmo trabalho, outra forma: em vez de programar o gateway, você descreve o que ele deve fazer.

KONG POR DENTRO



ONDE FICA A CONFIGURAÇÃO

Com banco (Postgres): muda na hora pela Admin API. É o que faremos ao vivo na docker-class.

Sem banco (declarativo): lê um kong.yml ao subir. Versionado no Git. É o do e-commerce.

Híbrido: um Kong de controle e vários de tráfego, para grandes instalações.

Escala: o Kong não guarda estado da requisição. Para aguentar mais tráfego, sobem mais réplicas atrás de um balanceador, todas com a mesma configuração.

Ponto único de falha: se o gateway cai, a loja inteira some para o cliente. Por isso, em produção, nunca roda uma réplica só.

Gratuito × pago: usamos o Kong de código aberto. A versão Enterprise acrescenta plugins (OIDC, cofres de segredo) e painéis.

PLUGINS: ONDE VALEM E EM QUE ORDEM

ONDE UM PLUGIN VALE

Global
todas as requisições (POST /plugins)

Service
tudo que vai para um serviço

Route
só aquela rota

Consumer
só para aquele cliente

O mais específico vence: um rate-limiting na rota passa por cima do mesmo plugin no service, que passa por cima do global.

ORDEM DE EXECUÇÃO (PRIORIDADE)

Plugin	Prioridade	Faz
pre-function	1.000.000	código antes de tudo
correlation-id	100.001	dá o X-Request-Id
cors	2.000	libera o navegador
jwt	1.450	quem é? (401)
key-auth	1.250	quem é? (401)
acl	950	pode? (403)
rate-limiting	910	quantas vezes? (429)
post-function	-1.000	código depois de tudo

A ordem tem lógica: primeiro descobre QUEM é (autenticação), depois se PODE (autorização), depois QUANTAS VEZES pode (limite). O acl precisa saber o consumer, por isso roda depois do jwt.

UMA LOJA DE PLUGINS

O Kong traz dezenas de plugins prontos, configurados por nome. Os destacados são os que vamos usar.

Categoria	Plugins	Para quê
Autenticação	key-auth ★ jwt ★ basic-auth oauth2 hmac-auth ldap-auth	descobrir quem está chamando
Segurança	acl ★ cors ip-restriction bot-detection request-size-limiting	decidir quem pode, de onde e quanto
Controle de tráfego	rate-limiting ★ proxy-cache request-termination	proteger os serviços de excesso
Transformação	correlation-id ★ request-transformer response-transformer	mexer em cabeçalhos, corpo e caminho
Observabilidade	file-log http-log prometheus opentelemetry	logs, métricas e rastreamento
Código próprio	pre-function ★ post-function ★	quando não existe plugin pronto

A lista completa: `docs.konghq.com/hub`. Antes de programar algo no gateway, procure se já existe um plugin.

AO VIVO: KONG DO ZERO

Na docker-class, o Kong sobe vazio e vamos registrando tudo pela Admin API, vendo o efeito a cada passo.

1. Service pedidos: ainda 404
2. Route /orders: 200 pela 8000
3. Service e route de pagamentos
4. Prefixo /api/pedidos e strip_path
5. correlation-id global
6. rate-limiting: a 6ª vira 429
7. key-auth e o primeiro consumer
8. jwt: consumer cliente; vencido, falso
9. acl: só o admin paga (403)
10. A porta dos fundos: 3003
11. Exportar para kong.yml
12. Recomeçar do zero

ROTEIRO: `docker-class/ROTEIRO-KONG.md`

PLUGIN CORRELATION-ID

```
POST localhost:8001/plugins
  name=correlation-id
  config.header_name=X-Request-Id
  config.generator=uuid
  config.echo_downstream=true
```

```
# no log do order-service:
GET /orders/1  | x-request-id=3c37be51-...
# na resposta ao cliente:
X-Request-Id: 3c37be51-...
```

O que faz: dá um id único para cada requisição e repassa no cabeçalho para o serviço.

Por que importa: com 5 serviços, um erro do cliente vira 5 logs separados. Com o mesmo id em todos, um grep encontra a história inteira.

echo_downstream: devolve o id ao cliente. Ele manda o id no chamado de suporte e você acha o problema.

Se o cliente já mandar um id, o Kong mantém o dele.

Escopo: global. Toda requisição precisa de um id, inclusive as recusadas.

PLUGIN RATE-LIMITING

```
POST localhost:8001/services/pedidos/plugins
  name=rate-limiting
  config.minute=5
  config.policy=local
  instance_name=limite-pedidos
```

```
HTTP/1.1 200 OK
X-RateLimit-Limit-Minute: 5
X-RateLimit-Remaining-Minute: 0
HTTP/1.1 429 Too Many Requests
{"message":"API rate limit exceeded"}
```

Janelas: second, minute, hour, day, month, year. Pode combinar: 10 por segundo e 1000 por hora.

Contar por quem? limit_by: consumer, ip, credential, header, path. Sem consumer, conta por IP.

Onde guarda a contagem: local (em cada réplica do Kong), cluster (no banco) ou redis (um lugar só para todas as réplicas).

O serviço nem fica sabendo: a 6ª requisição para no Kong.

No e-commerce: POST /auth/login aceita 10 tentativas por minuto por IP. É a defesa contra quem tenta adivinhar senhas (força bruta).

PLUGIN KEY-AUTH E OS CONSUMERS

```
# 1. proteger o service
POST /services/pedidos/plugins  name=key-auth
# 2. quem pode chamar
POST /consumers  username=app-parceiro
# 3. a chave dele
POST /consumers/app-parceiro/key-auth
     key=chave-secreta-123
```

```
GET /orders   apikey: chave-secreta-123
# o serviço recebe:
X-Consumer-Username: app-parceiro
```

Consumer é quem consome a API: um app, um sistema parceiro, uma pessoa. As credenciais pertencem a ele.

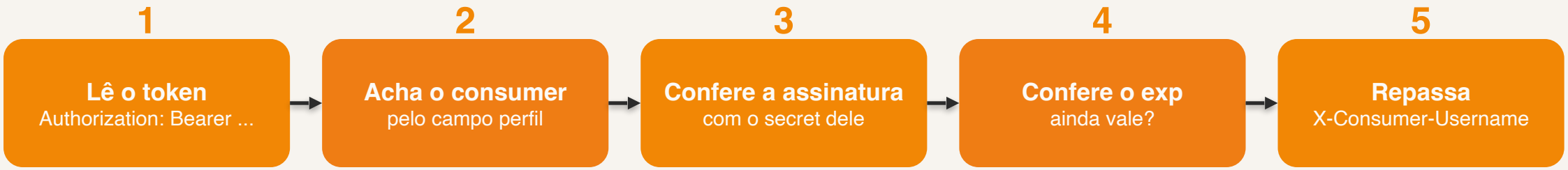
Sem chave ou chave errada: 401, e o serviço não recebe nada.

Identificação de graça: o Kong conta ao serviço quem chamou e permite limites por consumer.

`hide_credentials: true` apaga a chave antes de repassar: o serviço não precisa conhecê-la.

Quando usar: sistema chamando sistema, de fora (um marketplace, um parceiro). A chave não expira e não carrega dados: não serve para o login de pessoas.

PLUGIN JWT: COMO O KONG VALIDA



Tentativa	Resposta do Kong (401)
Sem token	"Unauthorized"
Token vencido	"token expired"
Assinado com outro segredo	"Invalid signature"
Perfil que ninguém cadastrou	"No credentials found for given 'perfil'"
Lixo no lugar do token	"Bad token; invalid JSON"

config.key_claim_name: qual campo do token aponta o consumer. O padrão é iss; nós usamos perfil.

config.claims_to_verify: [exp] obriga a validade. Sem isso, token velho passa.

HS256 × RS256: com HS256, o Kong guarda o mesmo segredo que assina. Com RS256, só a chave pública: ele confere, mas não consegue fabricar tokens.

O Kong não emite tokens. Quem emite é o serviço de usuários, no login. O Kong só confere se o token é legítimo e está no prazo.

PLUGIN ACL: GRUPOS E PERMISSÕES

```
POST /consumers/cliente/acls   group=cliente
POST /consumers/admin/acls     group=admin

POST /services/pagamentos/plugins
  name=acl
  config.allow[]=admin
```

```
# Ana (token com perfil cliente) tenta pagar
403 {"message":"You cannot consume this service"}
# Bia (token com perfil admin)
200 {"message":"Pagamento aprovado..."}
```

Precisa de autenticação antes: o acl decide pelo consumer, e quem descobre o consumer é o jwt ou o key-auth.

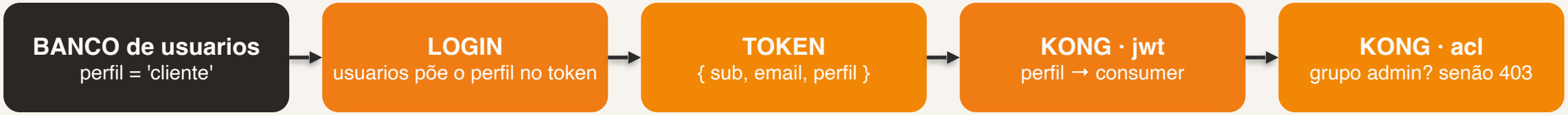
allow ou deny: lista de quem pode, ou de quem não pode.

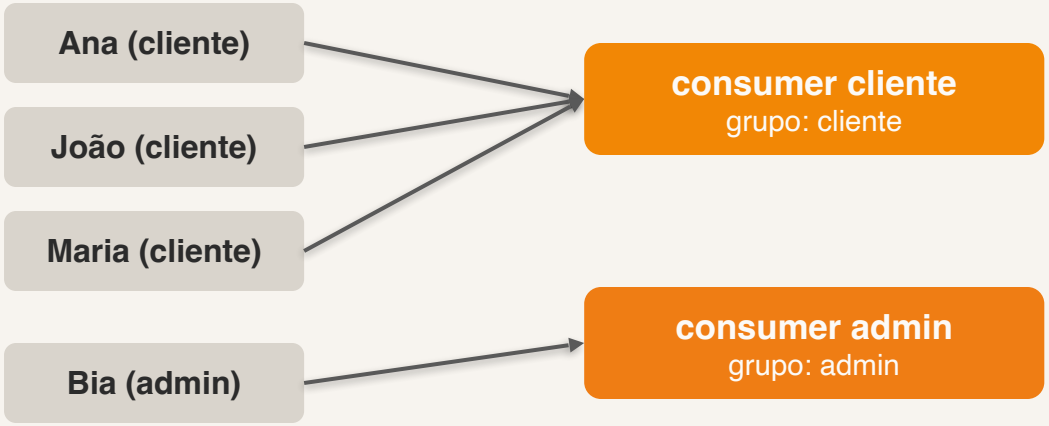
401 × 403: o jwt diz "não sei quem você é"; o acl diz "sei quem você é, e você não pode".

Repare: os consumers são perfis (cliente, admin), não pessoas. A Ana é só uma das clientes. O próximo slide explica por quê.

MILHARES DE USUÁRIOS, DOIS CONSUMERS

A pergunta: os clientes se cadastram sozinhos. Como o acl sabe quem é admin, se não existe um consumer para cada pessoa?





Os usuários não viram consumers. O consumer representa o perfil, que são poucos e fixos. O jwt acha o consumer pelo campo perfil do token (key_claim_name: perfil).

O Kong não abre o banco. Quem lê o perfil no banco é o serviço de usuários, no login. O token leva a informação até o Kong.

Mudou o perfil no banco? Vale no próximo login: o token antigo continua com o perfil antigo até vencer (2 h). Por isso tokens têm vida curta.

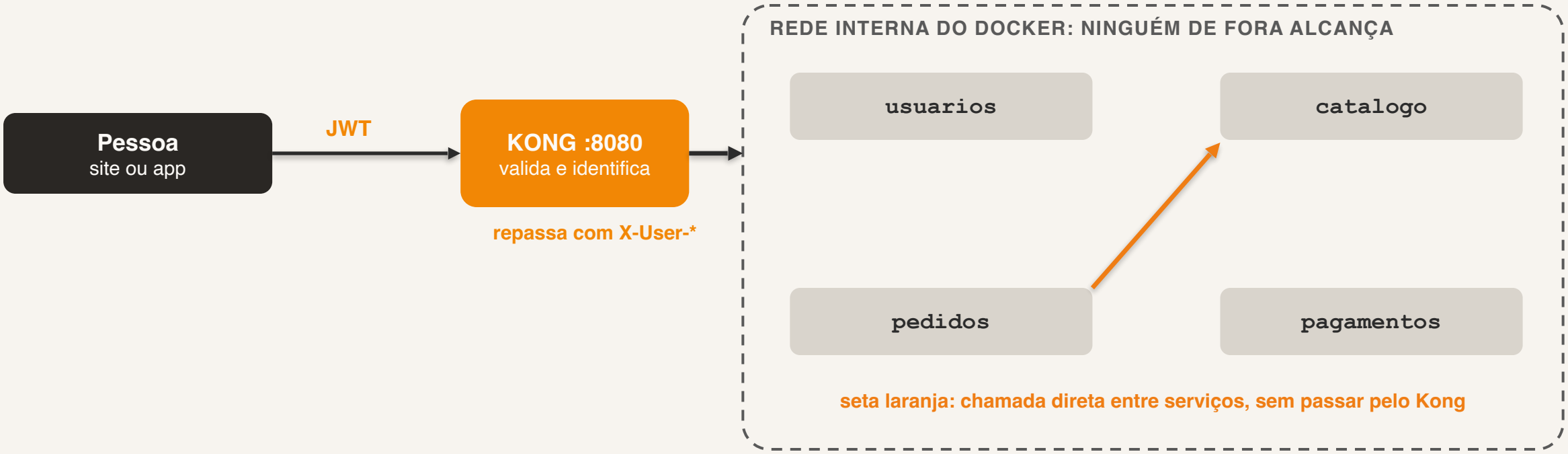
Por que não um consumer por usuário? O serviço de usuários teria que chamar a Admin API do Kong a cada cadastro, o Kong precisaria de banco, e um sumiço de sincronia deixaria alguém sem acesso.

QUAL USAR?

	key-auth	jwt	acl
Responde	quem é? (401)	quem é? (401)	pode? (403)
Credencial	chave fixa	token assinado	grupo do consumer
Expira?	não (até ser trocada)	sim (exp)	—
Carrega dados?	não	sim: id, e-mail, perfil	—
Quem cria	o admin, no Kong	o serviço de login	o admin, no Kong
Bom para	sistemas parceiros	pessoas logadas	poucos grupos fixos
No e-commerce	não usamos (só na demo)	rotas logado e admin	rotas admin: allow [admin]

Regra prática: pessoa faz login e recebe JWT; sistema externo recebe uma chave. Para o grupo, o acl.

BORDA × REDE INTERNA



Cliente → Kong: JWT emitido só por usuarios. O Kong valida, acha o consumer pelo perfil e confere o grupo.

Kong → serviço: o serviço confia nos X-User-* e não valida token. Ele só fala com quem está na rede interna.

Serviço → serviço (pedidos → catálogo; na docker-class, payment → order): direto, sem Kong, sem token. A proteção é não publicar porta.

E se não der para confiar na rede? Zero trust: cada serviço prova quem é com certificado (mTLS), em geral com um service mesh (Istio, Linkerd). Fica de fora do nosso projeto.

KONG.YML: SERVICES E ROUTES

```yaml
_format_version: "3.0"
services:
  - name: catalogo
    url: http://catalogo:3000
    routes:
      - name: vitrine
        methods: [GET]
        paths: ["~/(produtos|categorias)(/.*)?$"]
        strip_path: false
        tags: [publico]
      - name: gestao-catalogo
        methods: [POST, PUT, PATCH, DELETE]
        paths: ["~/(produtos|categorias)(/.*)?$"]
        strip_path: false
        tags: [admin]
        plugins: *so-admin    # jwt + acl
```

~ no início do caminho = expressão regular.

`strip_path: false` repassa o caminho inteiro. Com true, o Kong cortaria /produtos antes de encaminhar.

Mesmo caminho, métodos diferentes: GET é público, o resto exige admin.

A tag diz o nível de acesso, como o campo acesso do gateway do zero.

Rota que não casa com nada: 404 do próprio Kong.

KONG.YML: OS CONSUMERS

```
consumers:
  - username: cliente
    jwt_secrets:
      - { key: cliente, algorithm: HS256,
          secret: "__JWT_SECRET__" }
    acls: [{ group: cliente }]
  - username: admin
    jwt_secrets: [{ key: admin, ... }]
    acls: [{ group: admin }]
```

```
// usuarios, no login: o perfil vem do banco
jwt.sign({ sub: u.id, email: u.email, perfil: u.perfil },
  JWT_SECRET, { expiresIn: '2h' })
```

`key` da credencial = valor do campo perfil do token. Token de perfil "cliente" cai no consumer cliente.

`secret:` o mesmo JWT_SECRET de usuarios. __JWT_SECRET__ é trocado pela variável quando o container sobe.

`acls:` o grupo de cada consumer, que o plugin acl vai conferir.

Perfil inventado ("superadmin"): não existe consumer com essa key → 401.

Perfil novo no sistema (ex.: estoquista)? Um consumer e um grupo novos no kong.yml. É uma decisão de arquitetura, não um cadastro.

KONG.YML: JWT E ACL NAS ROTAS

```yaml
- name: listar-usuarios     # GET /usuarios
  tags: [admin]
  plugins: &so-admin
    - &jwt
      name: jwt
      config:
        key_claim_name: perfil
        claims_to_verify: [exp]
    - name: acl
      config: { allow: [admin] }

- name: meus-dados          # /usuarios/me
  tags: [logado]
  plugins: &logado [*jwt]

- name: gestao-catalogo     # POST /produtos...
  plugins: *so-admin

- name: login               # contra força bruta
  plugins: [{ name: rate-limiting,
    config: { minute: 10, limit_by: ip } }]
```

Três tipos de rota, três receitas:

publico: nenhum plugin de acesso.
logado: só jwt. Qualquer perfil com token válido passa (401 se não).
admin: jwt + acl. Token válido E grupo admin (403 se não).

&nome / *nome: recurso do YAML para nomear um trecho e reaproveitar. A receita de admin é escrita uma vez e usada em todas as rotas de admin.

Login: público, mas com limite: a 11ª tentativa no minuto recebe 429.

KONG.YML: PLUGINS GLOBAIS

```yaml
plugins:    # valem para todas as rotas
  - name: correlation-id
    config: { header_name: X-Request-Id, generator: uuid,
              echo_downstream: true }

  - name: pre-function    # antes de tudo
    config:
      access:
        - |
          kong.service.request.clear_header("X-User-Id")
          -- idem X-User-Email e X-User-Role

  - name: post-function   # depois do jwt e do acl
    config:
      access:
        - |
          local claims = jwt_parser:new(token).claims
          kong.service.request.set_header("X-User-Id", claims.sub)
          -- idem email e perfil
```

pre-function: apaga os X-User-* que o cliente mandou. Ninguém escolhe quem é.

post-function: copia id, e-mail e perfil do token para os cabeçalhos que os serviços leem.

Por que Lua? O jwt valida, mas não repassa os campos do token. O Kong gratuito não tem plugin pronto para isso.

Repare: a decisão de acesso não está no Lua. Quem decide é o acl.

O KONG NO DOCKER-COMPOSE.YML

```yaml
gateway:
  image: kong:3.9
  ports:
    - "8080:8000"   # a API da loja
    - "8001:8001"   # Admin API (leitura)
    - "8002:8002"   # Kong Manager
  volumes: [./kong:/kong:ro]
  entrypoint: ["sh", "-c", "sed ... && exec ... kong docker-start"]
  environment:
    KONG_DATABASE: "off"
    KONG_DECLARATIVE_CONFIG: /tmp/kong.yml
    KONG_PROXY_LISTEN: 0.0.0.0:8000
    KONG_ADMIN_LISTEN: 0.0.0.0:8001
    KONG_ADMIN_GUI_LISTEN: 0.0.0.0:8002
    KONG_UNTRUSTED_LUA_SANDBOX_REQUIRES: kong.plugins.jwt.jwt_parser
    JWT_SECRET: ${JWT_SECRET:-troque-este-segredo}
```

Imagem oficial, nenhum código nosso: só o kong.yml montado como volume.

Mudou o kong.yml? `docker compose restart gateway`

Para conferir: `curl localhost:8001/routes` ou o painel em localhost:8002.

E os serviços? Perdem o ports:. Só o gateway fica exposto.

DO ZERO × KONG

Passo	Gateway do zero (Node)	Kong (kong.yml)
Tabela de rotas	REGRAS + SERVICOS	services e routes com tags
X-Request-Id	primeiro middleware	plugin correlation-id
Apagar X-User-* de fora	delete req.headers[...]	plugin pre-function
Validar o JWT (401)	jwt.verify	plugin jwt + consumers
Rota de admin (403)	regra.acesso === 'admin'	plugin acl (consumer por perfil)
Repassar a identidade	req.headers['x-user-id']	plugin post-function (Lua)
Encaminhar	http-proxy-middleware	o próprio Kong (NGINX)
Rate limit, métricas, painel	teria que programar	plugins e Kong Manager prontos

A prova: os mesmos arquivos .http passam nos dois. Para o cliente, não importa qual gateway está na frente.

CADA SERVIÇO NA SUA LINGUAGEM?

Pode. O Kong não sabe, e não precisa saber, em que linguagem o serviço foi escrito. O que precisa ser respeitado são os contratos:

ROTAS HTTP + JSON

Os caminhos, os status codes e o formato { erro, detalhes } combinados entre os serviços.

X-USER-*

A identidade chega pronta. O serviço só lê X-User-Id, X-User-Email e X-User-Role.

O JWT

HS256, com sub, email, perfil, iss e exp. Só importa para quem emite: usuarios.

UM DOCKERFILE

Se sobe no compose e responde na porta combinada, serve. Java, Python, Go, C#, PHP.

O preço da liberdade: cada linguagem é mais uma imagem, mais um jeito de logar e mais coisa para a dupla manter. Liberdade, sim; dupla que usa quatro linguagens, só se souber defender.

PARTE 3: O SERVIÇO DE PEDIDOS

O terceiro serviço: carrinho e checkout, com o seu próprio Postgres.

É o primeiro serviço que depende de outro. Na hora da compra, ele precisa perguntar ao catálogo o preço de cada produto.

A PERGUNTA DA AULA: e se o catálogo não responder?

PEDIDOS: AS TABELAS (POSTGRES)

```
CREATE TABLE carrinho_itens (
  usuario_id UUID NOT NULL,      -- sem FK!
  produto_id TEXT NOT NULL,      -- id do Mongo
  quantidade INT NOT NULL CHECK (quantidade > 0),
  PRIMARY KEY (usuario_id, produto_id)
);
CREATE TABLE pedidos (
  id UUID PRIMARY KEY, usuario_id UUID, cliente_email TEXT,
  status TEXT CHECK (status IN ('AGUARDANDO_ESTOQUE',
    'AGUARDANDO_PAGAMENTO', 'PAGO', 'CANCELADO')),
  valor_total NUMERIC(12,2), endereco_entrega JSONB,
  forma_pagamento TEXT, cartao_final TEXT,  -- 4 dígitos
  criado_em TIMESTAMPTZ DEFAULT now()
);
CREATE TABLE itens_pedido (
  pedido_id UUID REFERENCES pedidos(id),  -- FK: mesmo banco
  produto_id TEXT, nome_produto TEXT,
  preco_unitario NUMERIC(12,2), quantidade INT
);
```

Sem FK para usuarios e produtos: eles moram em outros bancos. Quem garante que existem é o código, perguntando ao serviço dono.

Preço e nome copiados: se o produto mudar de preço amanhã, o pedido de hoje não muda. Não é duplicação, é histórico.

Cartão: só os 4 últimos dígitos. O número inteiro nunca é gravado.

Status: já nasce com os 4 estados. Quem faz o pedido andar entre eles é o Kafka, nas próximas aulas.

PEDIDOS: AS ROTAS

Método	Rota	Acesso	Retornos
GET	/carrinho	logado	200 { itens, total } com preços atuais do catálogo
POST	/carrinho/itens	logado	201 item adicionado · 400 quantidade inválida · 404 produto não existe
DELETE	/carrinho/itens/:produtoId	logado	204 removido · 404 não está no carrinho
POST	/pedidos	logado	202 pedido aceito · 400 carrinho vazio · 409 produto indisponível · 503 catálogo fora
GET	/pedidos	logado	200 meus pedidos
GET	/pedidos/:id	logado	200 pedido com itens · 404 não existe ou não é meu
POST	/pedidos/:id/cancelar	logado	200 cancelado · 409 status não permite
GET	/health	interno	200 serviço e banco ok (no gateway: /health/pedidos)

Por que 202 e não 201? O pedido foi aceito, mas ainda não terminou: falta reservar o estoque e cobrar. Hoje ele fica parado em AGUARDANDO_ESTOQUE; na próxima aula, o Kafka o faz andar.

CHECKOUT: PEDIDOS PERGUNTA AO CATÁLOGO

```
// pedidos/src/catalogo.js
async function buscarProdutos(ids, requestId) {
  try {
    const r = await fetch(
      `http://catalogo:3000/produtos?ids=${ids}`,
      { headers: { 'x-request-id': requestId },
        signal: AbortSignal.timeout(3000) });
    if (!r.ok) throw new Error(`status ${r.status}`);
    return await r.json();
  } catch {
    throw new CatalogoIndisponivel(); // vira 503
  }
}
```

Direto pela rede interna, sem passar pelo gateway: o gateway é a porta de fora.

Um pedido só para todos os produtos do carrinho (?ids=a,b,c), não um por produto.

Timeout de 3 s: sem ele, um catálogo travado trava o checkout junto.

Acoplamento temporal: para vender, o catálogo precisa estar no ar naquele instante. Catálogo fora → checkout responde 503, mas o carrinho e a lista de pedidos continuam funcionando.

Teste ao vivo: `docker compose stop catalogo` e tente finalizar uma compra.

PARA DISCUTIR EM DUPLA

1. pedidos deveria chamar o catálogo pelo gateway (localhost:8080)? O que ganharia e o que perderia?
2. O gateway caiu. O que para de funcionar? E se for o catálogo?
3. Por que o pedido copia o preço em vez de guardar só o produto_id?
4. Um aluno publicou a porta 3003 do pedidos "para testar". Qual o risco, agora que o serviço confia em X-User-*?
5. Onde vocês colocariam um limite de 10 requisições por segundo por cliente?

OS ARQUIVOS .HTTP AGORA

```
@baseUrl = http://localhost:8080        # uma URL só: a do gateway

### Login
# @name login
POST {{baseUrl}}/auth/login
Content-Type: application/json

{ "email": "cliente@loja.com", "senha": "cliente123" }

### Tentar se passar por admin (espera 401)
GET {{baseUrl}}/usuarios
X-User-Role: admin

### Cliente criando produto (espera 403)
POST {{baseUrl}}/produtos
Authorization: Bearer {{login.response.body.$.token}}
```

Novo arquivo: `http/pedidos.http` com o fluxo: login → adicionar ao carrinho → ver carrinho → checkout (202) → consultar o pedido. E os casos de erro: carrinho vazio, pedido de outro usuário, catálogo fora.

ROTAS PROTEGIDAS: USUARIOS

Método	Rota	Acesso	No Kong	Sem permissão
POST	/auth/login	público	rate-limiting (10/min por IP)	429 na 11ª tentativa
POST	/usuarios	público	—	—
GET · PUT	/usuarios/me	logado	jwt	401 sem token ou token inválido
GET · POST	/usuarios/me/enderecos	logado	jwt	401
DELETE	/usuarios/me/enderecos/:id	logado	jwt	401 · 404 se o endereço não é seu
GET	/usuarios	admin	jwt + acl allow [admin]	401 sem token · 403 perfil cliente
GET	/health/usuarios	público	—	—

Na aula passada, o próprio serviço validava o token nas rotas /me. Agora quem valida é o Kong; o serviço só lê X-User-Id.

O 404 do endereço continua sendo do serviço: o Kong sabe quem você é, mas não sabe de quem é cada endereço.

ROTAS PROTEGIDAS: CATALOGO

Método	Rota	Acesso	No Kong	Sem permissão
GET	/produtos?categoria=&busca=	público	—	—
GET	/produtos/:id	público	—	—
GET	/categorias	público	—	—
POST	/produtos	admin	jwt + acl allow [admin]	401 sem token · 403 perfil cliente
PUT	/produtos/:id	admin	jwt + acl allow [admin]	401 · 403
PATCH	/produtos/:id/estoque	admin	jwt + acl allow [admin]	401 · 403
DELETE	/produtos/:id	admin	jwt + acl allow [admin]	401 · 403
POST	/categorias	admin	jwt + acl allow [admin]	401 · 403

Na aula passada, as rotas de admin do catálogo ficaram abertas: qualquer um criava produto. Agora, cliente logado recebe 403 e quem não tem token recebe 401.

Mesmo caminho, métodos diferentes: GET /produtos é público; POST /produtos é admin. São duas routes no kong.yml.

O QUE ENTREGAR

1. **Kong sem banco**, com `kong/kong.yml` versionado. A 8080 é a única porta da API.
2. **Consumers cliente e admin** com grupos; jwt achando o consumer pelo perfil do token.
3. **Rotas fechadas:** logado com jwt (401); admin com jwt + acl (403). Acabou o catálogo aberto.
4. **correlation-id global e rate-limiting no login** (10 por minuto por IP).
5. **usuarios é o único que emite JWT**, com o perfil do banco. Os outros serviços leem X-User-*.
6. **Serviço pedidos:** 3 tabelas, carrinho e checkout chamando o catálogo direto, com timeout.
7. **Nenhuma porta publicada** além do gateway. Arquivos .http pelo gateway, mais o pedidos.http.

CHECKPOINT: tag `aula-gateway-pedidos` no repositório.

PRÓXIMA AULA

01 Entra o Kafka: o checkout publica o evento pedido.criado.

02 O catálogo escuta, reserva o estoque e responde com outro evento.

03 O pedido sai de AGUARDANDO_ESTOQUE sozinho, sem ninguém chamar ninguém.

04 Nasce o serviço de notificações, mandando e-mail pelo Mailpit.

Contato



Augusto Gehrke

https://www.linkedin.com/in/augusto-g-b54a5bb7

augusto.gehrke@amf.edu.br

augustogehrke@outlook.com